

Criptografia de Dados

Autor: Thiago de Mattos Azevedo Chaves

Data de Atualização: 16/07/2026

Status: Em Desenvolvimento

1. Visão Geral e Objetivo

Este módulo serve para proteger os dados (informações) sensíveis dos usuários da Plataforma SEP, com foco especial na privacidade e no cumprimento das leis de proteção de dados (como LGPD). E avaliar o impacto no custo de processamento (CPU) ao decriptar volumes de dados grandes.

Ele resolve o problema de segurança de dados no banco de dados. Caso aconteça uma invasão ou um vazamento de dados, as senhas e os documentos pessoais dos usuários estarão criptografados.

2. Pesquisa e Fundamentação (O que foi estudado)

Conceitos Chave

Hashing: O hashing transforma um texto em código (bytes) que não pode mais ser desfeito (ex: PBKDF2, usado pelo Django em senhas). Podemos pensar nele como uma via de mão única, só ida, sem volta. É a forma ideal para senhas, pois o sistema não precisa saber a senha, só comparar os códigos gerados.

Criptografia simétrica: É o método que usa uma chave secreta tanto para trancar quanto para destrancar os dados — nesse caso, pense como uma via de mão dupla, tem ida e volta. É o ideal para dados que o sistema precisa ler de volta, como CPF (no nosso caso, sobre o paciente é tudo: nome, endereço, telefone).

Fernet (biblioteca cryptography): esquema de criptografia simétrica autenticada, baseado em AES (Advanced Encryption Standard).

Custom Model Fields do Django: uso de `get_prep_value()` (chamado antes de gravar no banco) e `from_db_value()` (chamado ao ler do banco) para tornar a criptografia transparente ao restante da aplicação.

Bibliotecas e Ferramentas

- `django-cryptography`: ideia inicial, mais rejeitada, pois não funciona nas versões atuais do Django.
- `cryptography.fernet`: usada como base de um campo customizado próprio. Utilizada no código.

Decisões Arquiteturais (Prós e Contras)

Decisão: implementar um `EncryptedCharField` e `EncryptedTextField` próprio (`fields.py`), usando `cryptography.fernet` diretamente, em vez de depender de biblioteca terceira.

- **Prós:** sem risco de abandono de dependência crítica; mesma biblioteca de criptografia funcionaria em qualquer outro serviço da plataforma (ex: um serviço em FastAPI), padronizando a abordagem entre módulos; controle total sobre o comportamento de erro.

- Contras: responsabilidade de manutenção do código de criptografia passa a ser nossa, não de uma comunidade externa; exige mais rigor na revisão (qualquer bug na implementação afeta segurança diretamente).

Gargalo identificado — busca por conteúdo (icontains): campos criptografados não suportam busca parcial (LIKE e ILIKE), pois o valor armazenado é um token, diferente a cada gravação. Testado com `Prontuario.objects.filter(queixa_principal__icontains='ansiedade')` — retorna lista vazia mesmo havendo o termo no texto original.

Mitigação: campo auxiliar `tags_busca`, não criptografado, contendo palavras-chave relevantes extraídas ou atribuídas manualmente ao registro, usado exclusivamente para viabilizar buscas por conteúdo.

Outro gargalo identificado — busca exata (`__exact`): mesmo comparações de igualdade direta falham, pois o Fernet usa IV aleatório; o mesmo CPF gera tokens diferentes a cada gravação.

Mitigação recomendada: campo auxiliar `cpf_hash` (HMAC-SHA256, determinístico), indexado, usado para localizar o registro por igualdade. O campo `cpf` criptografado permanece como fonte da verdade do dado protegido; o hash serve apenas para localização.

3. Guia de Configuração e Instalação (Setup)

Dependências

```
pip install cryptography
pip install python-dotenv
pip install djangoRESTframework
```

O `djangoRESTframework` é necessário para a API REST de testes (seção 4.4) — sem ele os endpoints `/api/prontuarios/...` não funcionam.

Após instalar, adicionar `'rest_framework'` ao `INSTALLED_APPS` no `settings.py`, junto com o app onde está o `models.py`:

```
INSTALLED_APPS = [
    ...
    'rest_framework',
    'core',
]
```

Variáveis de Ambiente (.env)

Criar o arquivo `.env` na raiz do projeto. Esse arquivo NUNCA deve ser commitado — coloca ele no `.gitignore`.

```
FIELD_ENCRYPTION_KEY=(aqui cole sua chave)
```

Gerar uma chave nova (executar apenas uma vez por ambiente; trocar a chave torna ilegíveis os dados já criptografados):

```
python manage.py shell
>>> from cryptography.fernet import Fernet
>>> Fernet.generate_key()
```

Configuração no settings.py

```
import os
from pathlib import Path
```

```

from dotenv import load_dotenv

BASE_DIR = Path(__file__).resolve().parent.parent
load_dotenv(BASE_DIR / ".env")

FIELD_ENCRYPTION_KEY = os.environ.get("FIELD_ENCRYPTION_KEY")

if not FIELD_ENCRYPTION_KEY:
    raise ValueError(
        "FIELD_ENCRYPTION_KEY nao encontrada. Verifique o .env."
    )

```

Comandos Específicos

```

python manage.py makemigrations core
python manage.py migrate

```

4. Implementação e Casos de Uso (Como Usar)

4.1. Campo de criptografia customizado (core/fields.py)

Implementação do campo de criptografia, usada como base para os models. `get_prep_value()` criptografa antes de gravar; `from_db_value()` descriptografa ao ler — o processo é transparente ao restante da aplicação.

```

from cryptography.fernet import Fernet, InvalidToken
from django.conf import settings
from django.db import models

def get_fernet() -> Fernet:
    key = settings.FIELD_ENCRYPTION_KEY
    if isinstance(key, str):
        key = key.encode()
    return Fernet(key)

class EncryptedFieldMixin:
    def get_internal_type(self):
        return "TextField"

    def get_prep_value(self, value):
        value = super().get_prep_value(value)
        if value is None or value == "":
            return value
        token = get_fernet().encrypt(str(value).encode())
        return token.decode()

    def from_db_value(self, value, expression, connection):
        if value is None or value == "":
            return value
        try:
            return get_fernet().decrypt(value.encode()).decode()
        except InvalidToken:
            raise ValueError(
                "Nao foi possivel descriptografar o campo."
            )

```

```

    def to_python(self, value):
        return value

class EncryptedCharField(EncryptedFieldMixin, models.CharField):
    pass

class EncryptedTextField(EncryptedFieldMixin, models.TextField):
    pass

```

4.2. Definições no Model (core/models.py)

```

from django.db import models
from core.fields import EncryptedCharField, EncryptedTextField

class Prontuario(models.Model):
    paciente_nome = EncryptedCharField(max_length=100)
    cpf = EncryptedCharField(max_length=11)

    queixa_principal = EncryptedTextField()
    evolucao_queixa = EncryptedTextField()
    historico_tratamento = EncryptedTextField()

    # Campo nao criptografado, de proposito, para viabilizar busca
    tags_busca = models.CharField(max_length=255, blank=True)

    criado_em = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"Prontuario de {self.paciente_nome}"

```

4.3. Exemplos Práticos de Uso

Exemplo 1 — Salvando um prontuário criptografado:

```

from core.models import Prontuario

Prontuario.objects.create(
    paciente_nome="Maria da Silva",
    cpf="12345678900",
    queixa_principal="Paciente relata episodios de ansiedade.",
    evolucao_queixa="Segunda sessao, melhora parcial.",
    historico_tratamento="Iniciou terapia em 2024.",
    tags_busca="ansiedade",
)

```

Leitura via ORM (descriptografia automática e transparente):

```

p = Prontuario.objects.get(id=1)
print(p.paciente_nome) # "Maria da Silva"

```

Exemplo 2 — Buscando por conteúdo (usando o campo auxiliar não criptografado):

```

# NAO funciona (campo criptografado, retorna lista vazia):
Prontuario.objects.filter(queixa_principal__icontains='ansiedade')

# Funciona (campo auxiliar nao criptografado):
Prontuario.objects.filter(tags_busca__icontains='ansiedade')

```

Exemplo 3 — Mascarando dados para exibição (core/masking.py):

```
import re

def mask_cpf(cpf: str) -> str:
    digits = re.sub(r"\D", "", cpf or "")
    if len(digits) != 11:
        return "***.***.***-**"
    return f"***.***.{digits[6:9]}-{digits[9:]}"

# uso:
p = Prontuario.objects.get(id=1)
print(mask_cpf(p.cpf)) # "***.***.789-00"
```

4.4. API REST de teste (core/views.py, core/serializers.py, core/urls.py)

Para facilitar testes manuais (Thunder Client, Postman, curl) sem depender do Django shell, foi criada uma API REST simples com Django REST Framework, cobrindo CRUD completo mais dois endpoints auxiliares.

GET	/api/prontuarios/	-> lista (dados descriptografados)
POST	/api/prontuarios/	-> cria (criptografa automaticamente)
GET	/api/prontuarios/{id}/	-> detalhe
PUT	/api/prontuarios/{id}/	-> atualiza tudo
PATCH	/api/prontuarios/{id}/	-> atualiza parcial
DELETE	/api/prontuarios/{id}/	-> remove
GET	/api/prontuarios/buscar?q=	-> busca via tags_busca
GET	/api/prontuarios/mascarado/	-> lista com nome/CPF mascarados

Instalação: pip install djangorestframework, adicionar 'rest_framework' ao INSTALLED_APPS, e incluir as rotas do app no urls.py do projeto.

O endpoint /mascarado/ usa um serializer separado (ProntuarioMascaradoSerializer) que aplica mask_cpf e mask_nome na saída, sem alterar o dado real armazenado — simula uma tela ou relatório de baixo privilégio.

5. Testes e Validação

5.1. Teste de inserção e leitura via ORM

Criado um registro de teste via Django shell (Prontuario.objects.create(...)) e confirmado que a leitura via ORM (Prontuario.objects.get(id=...)) retorna os valores originais corretamente — nome, CPF e textos de prontuário legíveis — comprovando que a criptografia é transparente para a aplicação.

5.2. Teste de ilegibilidade direta no banco

Consulta executada diretamente no banco (SQLite local; equivalente a psql/DBBeaver em Postgres/MySQL), sem passar pelo Django:

```
SELECT * FROM core_prontuario;
```

Resultado observado — captura real de uma linha do banco. Os campos criptografados retornam tokens Fernet ilegíveis; o campo tags_busca, propositalmente não criptografado, retorna texto puro; id e criado_em, campos técnicos, não são criptografados:

Campo	Valor bruto no banco
-------	----------------------

id	1
paciente_nome	gAAAAABqWZ0cTNZgg0RGMXoFsoVHcobXN7NvoigJkwhdpsb_eI... (token Fernet)
cpf	gAAAAABqWZ0dKEE8ACZvtVQz2ybCX70M_1TXn0446dtEjPUerC... (token Fernet)
queixa_principal	gAAAAABqWZ0dLYkBTXp6NNSuTNFxy7pmaJT1of4kJP6aANnDf... (token Fernet)
evolucao_queixa	gAAAAABqWZ0dyAJ_by8Ia-FfTs5l7yx0LSRn6T0sqThQXcU5y9... (token Fernet)
historico_tratamento	gAAAAABqWZ0dL71bZOatcirRdu5F1YEi3r3zdU-K9-0qM0aIqi... (token Fernet)
tags_busca	ansiedade (texto puro – de proposito)
criado_em	2026-07-17 02:29:49.004345

5.3. Teste de limitação de busca

Confirmado que `Prontuario.objects.filter(queixa_principal__icontains='ansiedade')` retorna lista vazia mesmo havendo o termo no conteúdo original, e que `Prontuario.objects.filter(tags_busca__icontains='ansiedade')` retorna corretamente o registro esperado — validando a estratégia de mitigação da seção 2.

5.4. Teste de compatibilidade de versão

Confirmado que a biblioteca `django-cryptography` falha ao migrar em Django 6.0.7 (`ImportError` relacionado a `django.utils.baseconv`, módulo removido nas versões recentes do Django). O campo customizado próprio (`fields.py`) foi testado com sucesso em Django 6.0.7, sem dependências externas além de `cryptography` e `python-dotenv`.

5.5. Teste de desempenho (custo de CPU ao decriptar em volume)

Populado o banco com 1.000 e 5.000 registros (5 campos criptografados cada) via `bulk_create`, e medido o tempo de leitura completa (acessando todos os campos criptografados) contra uma leitura equivalente sem decriptação (`.only()` em campo não criptografado):

Volume	Com decriptação (5 campos)	Sem decriptação (.only)	Overhead
1.000 registros	0,1552s	0,0097s	~15,7x mais lento
5.000 registros	0,7572s	—	~0,151ms / registro (linear)

Conclusão: o custo cresce de forma linear e estável (~0,15ms por registro com 5 campos criptografados), sem degradação desproporcional em volume. Para telas de listagem paginadas (20-50 registros), o overhead é irrelevante (menos de 10ms). O impacto real aparece em exportações/relatórios sem paginação ou endpoints que retornam grandes volumes de uma vez — nesses casos, recomenda-se usar `.only()` para não carregar/decriptar campos criptografados que não serão exibidos.

5.6. Teste dos endpoints da API

Servidor local (`python manage.py runserver`) testado com requisições reais via `curl`, simulando o uso via Thunder Client/Postman:

- `POST /api/prontuarios/` — criado registro de teste, retornou 201 com dados já descriptografados na resposta (`paciente_nome`, `cpf`, `queixa_principal` legíveis).
- `GET /api/prontuarios/` e `/api/prontuarios/{id}/` — retornam os registros corretamente, descriptografados.
- `GET /api/prontuarios/buscar/?q=insonia` — retornou corretamente os registros com essa tag em `tags_busca`.

- GET /api/prontuarios/mascarado/ — retornou nome e CPF mascarados na resposta, ex: "paciente_nome": "Joao P*****", "cpf": "****.***.321-00", confirmando que o mascaramento é aplicado apenas na camada de serialização, sem alterar o dado armazenado.

6. Referências e Links Úteis

- Documentação oficial da biblioteca cryptography (Fernet): <https://cryptography.io/en/latest/fernet/>
- Django — Custom Model Fields (get_prep_value / from_db_value): <https://docs.djangoproject.com/en/stable/howto/custom-model-fields/>
- Django — Password management / PBKDF2: <https://docs.djangoproject.com/en/stable/topics/auth/passwords/>
- Django REST Framework — documentação oficial: <https://www.django-rest-framework.org/>
- Vídeo sobre hash: <https://www.youtube.com/watch?v=hcUBjPo5-jk>
- DBeaver — download: <https://dbeaver.io/download/>